

□ Home Assistant YAML Konfigurációs Útmutató

Átfogó referencia full stack fejlesztőknek

Verzió: 2026. május
HA Core 2025.x / 2026.x alapján

Összeállította: Hermes Agent — Végh Tibor részére

Tartalomjegyzék

1. Bevezetés — Miért és mikor használj YAML-t?
2. YAML Szintaxis Alapok (JavaScript fejlesztőknek)
3. A configuration.yaml — A központi konfigurációs fájl
4. YAML Fájlok és Fájlszervezés
5. secrets.yaml — Titkok tárolása
6. Entity Customization
7. Packages (Csomagok)
8. Automatizációk YAML-ben
9. Feltételek (Conditions)
10. Script-ek YAML-ben
11. Szcnák (Scenes) YAML-ben
12. Template Sensor-ok YAML-ben
13. Gyakori YAML Integráció Példák
14. Hibakeresés és Validáció
15. YAML Eszközök és Validátorok
16. YAML vs UI — Döntési Mátrix
17. Fontos Linkek
18. Zárszó — Best Practices

1. Bevezetés — Miért és mikor használj YAML-t?

A Home Assistant kétféle konfigurációs módot támogat: grafikus felhasználói felület (UI) és kézi YAML szerkesztés. Mindkettőnek megvan a maga helye — az alábbiakban összefoglaljuk, mikor érdemes melyiket választani.

□ Mikor használj YAML-t:

- Speciális, nem UI-barát integrációk — pl. `template sensor`, `template binary_sensor`, `customize`, `mqtt` egyedi eszközök
- Komplex automatizációk amik UI-ban túl körülményesek (pl. mélyen egymásba ágyazott `choose`, `repeat`, `parallel`)
- Szcenáriók (scenes) — határozottan gyorsabb YAML-ben megírni
- Csomagok (packages) — logikusan elkülönített eszközcsoportok
- Egyedi `input_*` helperek deklarálása batch-szerűen
- Verziókezelés — YAML fájlokat lehet Git-ben tárolni
- Template-ek (Jinja2) a `configuration.yaml`-ben — csak YAML-ből érhetők el teljes körűen
- Secrets kezelés — a `secrets.yaml` fájlba rejthetők a jelszavak/API kulcsok

□ Mikor kerül a YAML-t:

- Beépített integrációk amiket a UI Settings → Integrations alatt támogat (ezek UI-ból karbantarthatók)
- Egyszerű automatizációk (1 trigger + 1 action) — a UI automata generálja az `automations.yaml`-t, kézzel ne nyúlj hozzá
- UI által menedzselt fájlok — `automations.yaml`, `scenes.yaml`, `scripts.yaml` — ezeket a UI írja/felülírja

2. YAML Szintaxis Alapok (JavaScript fejlesztőknek)

Alapfogalmak

JavaScript	YAML megfelelője
<code>{ kulcs: ertekek }</code>	kulcs: ertekek (mapping)
<code>[elem1, elem2]</code>	- elem1 / - elem2 (list/sequence)
<code>{ a: { b: c } }</code>	a: / ..b: c (beljebb húzva, 2 space)
<code>// komment</code>	# komment
indentáció (bármilyen)	2 space — TILOS a TAB!

Kritikus szabályok

```
# HELYES – 2 space indentáció:
sensor:
  - platform: mqtt
    state_topic: "sensor/topic"
```

```
# HELYTELEN – TAB használata:
sensor:
  - platform: mqtt    # ← HIBA: "found character '\t'"
```

Booleans — A leggyakoribb buktató

YAML automatikusan értelmezi ezeket true-ként: `yes`, `Y`, `true`, `True`, `on`, `On`, `ON` — és false-ként: `no`, `N`, `false`, `False`, `off`, `Off`, `OFF`

Ha stringként akarod az "on" vagy "off" szavakat (pl. entity state), IDÉZŐJELEZD!

```
# HELYES – stringként kezelődik
state: "on"
```

```
# HELYTELEN – YAML true-ként értelmezi
state: on
```

Duplikált kulcsok

Ha ugyanaz a kulcs kétszer szerepel, az utolsó nyer — nincs merge!

```
sensor:
  - platform: template
sensor:    # ← FELÜLÍRJA az előzőt!
  - platform: mqtt
```

⚠ A HA case-sensitive: `light.Living_Room` ≠ `light.living_room`. A pontos nevet a Developer Tools → States-ban ellenőrizd.

3. A configuration.yaml — A központi konfigurációs fájl

Helye:

- HA OS (Raspberry Pi): `/config/configuration.yaml`
- HA Core (manuális): a HA config mappában
- HA Container: `/config/configuration.yaml`

Minimális struktúra:

```
homeassistant:
  name: "Otthon"
  latitude: 47.8
  longitude: 19.9
  elevation: 200
  unit_system: metric          # vagy: us_customary
  time_zone: "Europe/Budapest"
  currency: HUF
  country: HU

default_config:                # Betölti az összes alap integrációt

automation: !include automations.yaml
script:      !include scripts.yaml
scene:       !include scenes.yaml

sensor:       !include sensors.yaml
binary_sensor: !include binary_sensors.yaml
switch:       !include switches.yaml
light:        !include lights.yaml
template:     !include templates.yaml
```

A homeassistant: blokk kulcsai:

Kulcs	Típus	Leírás
name	string	Az instance neve
latitude` / `longitude	float	Hely koordináták (naplemente/hajnal számításhoz)
elevation	integer	Tengerszint feletti magasság (méter/láb)
unit_system	metric / us_customary	Mértékegység rendszer
time_zone	string	IANA timezone (pl. Europe/Budapest)
currency	string	Pénznem ISO kód (pl. HUF)
country	string	Országkód (ISO 3166-1 alpha-2)
customize	map	Entity specifikus beállítások
customize_domain	map	Domain-wide beállítások
packages	map	Logikai eszközcsoporthok (lásd 7. fejezet)
allowlist_external_dirs	list	Engedélyezett külső mappák
allowlist_external_urls	list	Engedélyezett külső URL-ek
media_dirs	map	Média mappák aliassal

4. YAML Fájlok és Fájlszervezés

!include — Fájl betöltése

```
# configuration.yaml
sensor: !include sensors.yaml

# sensors.yaml
- platform: template
  sensors:
    heti_nap:
      value_template: "{{ now().strftime('%A') }}"
```

Az include-olt fájlban NE ismételd a szülő kulcsot — a !include a tartalmat helyettesíti be.

!include_dir_list — Mappa betöltése listaként

```
automation: !include_dir_list automations/

/config/automations/
├─ welcome_home.yaml
├─ night_mode.yaml
└─ away_mode.yaml
```

Minden .yaml fájl a mappában egy listaelem lesz.

!include_dir_named — Mappa betöltése név-érték párokként

```
input_boolean: !include_dir_named input_booleans/

/config/input_booleans/
├─ guest_mode.yaml → kulcs: guest_mode
└─ cleaning.yaml → kulcs: cleaning
```

A fájlnev lesz a kulcs, a tartalom az érték. Legtöbbet ezt érdemes használni!

!include_dir_merge_list — Mappa listáinak összefésülése

```
automation manual: !include_dir_merge_list automations/
automation ui: !include automations.yaml
```

Minden fájl egy-egy listát tartalmaz, az összes lista összefésülődik.

!include_dir_merge_named — Mappa mappingjeinek összefésülése

Minden fájl kulcs-érték párokat tartalmaz, mind összeolvad.

5. secrets.yaml – Titkok tárolása

Soha ne írd jelszavakat, API kulcsokat közvetlenül a configuration.yaml-be!

```
# secrets.yaml
wifi_ssid: "MyWiFi"
wifi_password: "SuperSecret123"
mqtt_broker: 192.168.1.100
mqtt_password: "mqtt_secret"

# configuration.yaml – hivatkozás !secret-tel
sensor:
  - platform: mqtt
    broker: !secret mqtt_broker
    password: !secret mqtt_password
```

i Secrets feloldási sorrend: A HA először a fájl melletti secrets.yaml-t nézi, majd a configuration.yaml mellett, végül a gyökér mappában lévő.

Debug: Add hozzá a `logger: debug`-t a secrets.yaml tetejére, hogy lásd, hol töltődnek be a titkok (a titok értéke nem kerül a logba).

6. Entity Customization

Entity-ek megjelenésének, nevének, ikonjának testreszabása:

```
homeassistant:
  customize:
    light.nappali_fo:
      friendly_name: "Nappali fővilágítás"
      icon: mdi:ceiling-light
      assumed_state: false
    sensor.kulso_homerseklet:
      friendly_name: "Külső hőmérséklet"
      unit_of_measurement: "°C"
      icon: mdi:thermometer

  customize_domain:
    light:
      icon: mdi:lightbulb
    switch:
      icon: mdi:toggle-switch
```

Elérhető customize kulcsok:

Kulcs	Leírás
friendly_name	Emberi olvasható név
icon	MDI ikon (pl. mdi:thermometer)
picture	Kép URL az entity-hez
assumed_state	Ha true, a UI nem jelzi ha bizonytalan az állapot
unit_of_measurement	Mértékegység felülírása
device_class	Eszközosztály (pl. temperature, humidity, motion)
entity_picture	Entity képének URL-je

7. Packages (Csomagok)

A packages a legerősebb módja a konfiguráció logikai csoportosításának. Egy csomag = egy eszköz/logikai egység teljes konfigurációja (sensorok, switchek, automatizációk, scriptek egy helyen).

Alapszintaxis:

```
homeassistant:
  packages:
    nappali_vilagitas:
      switch:
        - platform: mqtt
          name: "Nappali főkapcsoló"
          command_topic: "nappali/fokapcsoló/set"
      automation:
        - alias: "Nappali világítás este"
          triggers:
            - trigger: sun
              event: sunset
          actions:
            - action: switch.turn_on
              target:
                entity_id: switch.nappali_fokapcsoló
```

Packages mappa használata (ajánlott):

```
# configuration.yaml
homeassistant:
  packages: !include_dir_named packages/

/config/packages/
├─ nappali_vilagitas.yaml
├─ halo_vilagitas.yaml
├─ kert_loccs.yaml
├─ garazs_kapu.yaml
└─ riaszto.yaml

# packages/nappali_vilagitas.yaml
switch:
  - platform: mqtt
    name: "Nappali főkapcsoló"
    command_topic: "..."
automation:
  - alias: "Nappali este be"
    triggers: [...]
    actions: [...]
```

Merge szabályok:

Típus	Merge-elhető?	Megjegyzés
Platform-alapú (light:, switch:, sensor:)	☑ Igen	Lista szinten merge-elve
Entity ID-s (input_boolean:, input_number:)	☑ Igen, ha unique ID	Kulcsok nem ütközhetnek
Egyéb dictionary-k	⚠ Csak ha unique	Egy kulcs csak egyszer

⚠ auth_providers NEM lehet package-ben — azt a HA még a packages betöltése előtt feldolgozza.

8. Automatizációk YAML-ben

Alapstruktúra:

```
automation:
- alias: "Felkapcsolás naplementekor"
  id: "a1b2c3d4-e5f6-7890-abcd-ef1234567890" # UUID a trace-ekhez
  description: "Ha lenyugszik a nap és itthon vagyunk"
  initial_state: true
  mode: single # single | restart | queued | parallel
  max: 10 # csak queued/parallel módban
  max_exceeded: warning
  triggers:
    - trigger: sun
      event: sunset
      offset: "-01:00:00"
    - trigger: state
      entity_id: person.tibor
      to: "home"
  conditions:
    - condition: time
      after: "16:00:00"
      before: "23:00:00"
  actions:
    - action: light.turn_on
      target:
        entity_id: group.nappali_vilagitas
```

Automatizáció módok:

Mode	Viselkedés
single	Ha már fut, az új triggert eldobja (warning)
restart	Megállítja a futót, elindítja az újat
queued	Sorba állítja, egyesével futnak le (max: N)
parallel	Párhuzamosan futtat több példány (max: N)

Összes trigger típus:

Trigger	Leírás
<code>trigger: state</code>	Entity állapotváltozásra (from:, to:, for:)
<code>trigger: mqtt</code>	MQTT topic-ra érkező üzenetre
<code>trigger: time</code>	Adott időpontban (at: "16:30:00")
<code>trigger: time_pattern</code>	Ismétlődő időzítés (pl. minden 5 perc)
<code>trigger: homeassistant</code>	HA eseményekre (start, stop)
<code>trigger: event</code>	Egyedi eseményre (event_type: my_event)
<code>trigger: sun</code>	Nap eseményre (sunset, sunrise + offset)
<code>trigger: zone</code>	Zóna belépés/kilépés
<code>trigger: tag</code>	NFC tag scannelés
<code>trigger: conversation</code>	Assist hangparancsra
<code>trigger: numeric_state</code>	Numerikus állapot küszöbértékre
<code>trigger: calendar</code>	Naptár eseményre
<code>trigger: geo_location</code>	Geolokációs eseményre
<code>trigger: persistent_notification</code>	Értesítés megjelenésekor
<code>trigger: template</code>	Jinja2 template true értékére
<code>trigger: webhook</code>	Webhook hívásra
<code>trigger: device</code>	UI-ból konfigurált device trigger

9. Feltételek (Conditions)

A conditions lista alapértelmezésben AND kapcsolatban van.

Összes condition típus:

Condition	Leírás	Példa
condition: state	Entity állapotának ellenőrzése	state: "on"
condition: numeric_state	Numerikus érték vizsgálata	below: 25
condition: time	Időablak vizsgálat	after: "08:00"
condition: template	Jinja2 template	{{ ... int < 20 }}
condition: sun	Nap állása	after: sunset
condition: zone	Zóna ellenőrzés	zone: zone.home
condition: trigger	Trigger ID ellenőrzés	id: mozgás_ert
condition: and	Logikai AND csoport	conditions: [...]
condition: or	Logikai OR csoport	conditions: [...]
condition: not	Logikai NOT	conditions: [...]

Logikai feltételek kombinálása:

```
conditions:
- condition: and
  conditions:
    - condition: state
      entity_id: binary_sensor.mozgas
      state: "on"
    - condition: or
      conditions:
        - condition: state
          entity_id: light.nappali
          state: "off"
        - condition: numeric_state
          entity_id: sensor.fenyero
          below: 50

# Shorthand template condition:
conditions:
- "{{ is_state('device_tracker.iphone', 'home') }}"
```

10. Script-ek YAML-ben

A script egy előre meghívható lépéssorozat. Különbség az automatizációtól: automatizáció = trigger hatására indul, script = meghívásra indul.

```
script:
  jotekony_vilagitas:
    alias: "Jó éjszakát világítás"
    icon: mdi:weather-night
    description: "Lassan lehalványítja a lámpákat"
    fields:
      sebesseg:
        name: "Sebesség"
        description: "Hány másodperc alatt halványuljon"
        required: false
        default: 30
        selector:
          number:
            min: 5
            max: 120
    variables:
      lepes_ido: "{{ sebesseg | int // 10 }}"
    sequence:
      - action: light.turn_on
        target:
          entity_id: light.halo_fo
        data:
          brightness_pct: 50
      - delay:
          seconds: "{{ lepes_ido }}"
      - action: light.turn_off
        target:
          entity_id: light.halo_fo
```

Script action típusok:

Action	Leírás
action:	Service hívás (target, data, data_template)
delay:	Várakozás (seconds, HH:MM:SS, vagy kulcs: érték)
wait_template:	Várakozás amíg egy template true nem lesz
wait_for_trigger:	Várakozás amíg egy trigger be nem következik
variables:	Változók beállítása
if: / then: / else:	Feltételes végrehajtás
choose: / default:	Többágú feltétel (switch/case)
repeat:	Ciklus (while, until, count)
parallel:	Párhuzamos action-ök
event:	Esemény kiváltása
scene:	Scene aktiválása (rövidítés)
stop:	Script leállítása (reason:, error: true/false)

Choose (switch/case):

```
- choose:
  - conditions:
    - condition: sun
      after: sunset
    sequence:
    - action: light.turn_on
      data: {}
  default:
    - action: light.turn_off
      data: {}
```

Repeat (loop):

```
- repeat:
  count: 4
  sequence:
  - action: light.turn_on
    target:
      entity_id: "{{ 'light.lampa_' ~ repeat.index }}"
```


11. Szcénák (Scenes) YAML-ben

A scene = entity-ek előre beállított állapotainak halmaza.

```
# scenes.yaml
- id: "szcena_esti"
  name: "Esti hangulat"
  icon: mdi:weather-night
  entities:
    light.nappali_fo:
      state: "on"
      brightness: 60
      color_temp: 350
    light.halo_fo:
      state: "off"
    cover.nappali_fuggony:
      state: "on"          # "on" = zárt (cover domain)
    light.etkezo:
      state: "on"
      brightness: 40
      rgb_color: [255, 120, 50]
```

12. Template Sensor-ok YAML-ben

Az egyik leghasznosabb YAML-only feature:

```
# templates.yaml – trigger-alapú:
- trigger:
  - trigger: time_pattern
    hours: "*"
    minutes: "*"
  - trigger: homeassistant
    event: start
  sensor:
  - name: "Kinti idő összefoglaló"
    unique_id: kinti_ido_osszefoglalo
    state: >
      {% set temp = states('sensor.kulso_homerseklet') %}
      {% set hum = states('sensor.kulso_paratartalom') %}
      Kint {{ temp }}°C, páratartalom {{ hum }}%
    attributes:
      temperature: "{{ states('sensor.kulso_homerseklet') }}"
      last_updated: "{{ now() }}"

# Adatvezérelt template binary sensor:
- binary_sensor:
  - name: "Ajtó nyitva"
    unique_id: ajto_nyitva
    device_class: door
    state: >
      {{ is_state('binary_sensor.bejarati_ajto_kontakt', 'on') }}
```

Elérhető Template függvények (Jinja2):

Függvény	Leírás
<code>states('entity_id')</code>	Entity állapotának lekérése (string)
<code>is_state('entity_id', 'state')</code>	Boolean ellenőrzés
<code>state_attr('entity_id', 'attr')</code>	Attribute lekérése
<code>float()</code> , <code>int()</code>	Típuskonverzió
<code>now()</code> , <code>utcnow()</code>	Aktuális dátum/idő
<code>as_timestamp()</code>	Dátum → Unix timestamp
<code>relative_time()</code>	Relatív idő (pl. "2 hours ago")
<code>expand()</code>	Group-ok kibontása tagokra
<code>area_id()</code> , <code>area_name()</code>	Terület információk
<code>device_id()</code> , <code>device_attr()</code>	Eszköz információk
<code>iif(cond, if_true, if_false)</code>	Inline if/else
<code>regex_match()</code> , <code>regex_replace()</code>	Regex műveletek
<code>strptime()</code>	String → datetime parse

13. Gyakori YAML Integráció Példák

MQTT sensor:

```
sensor:
  - platform: mqtt
    name: "Kültéri hőmérséklet"
    state_topic: "sensor/kulter/homerseklet"
    unit_of_measurement: "°C"
    device_class: temperature
    value_template: "{{ value_json.temperature }}"
```

MQTT switch:

```
switch:
  - platform: mqtt
    name: "Garázs főkapcsoló"
    command_topic: "garazs/fokapcsoló/set"
    state_topic: "garazs/fokapcsoló/state"
    payload_on: "ON"
    payload_off: "OFF"
    optimistic: false
    qos: 1
    retain: true
```

REST sensor:

```
sensor:
  - platform: rest
    name: "Árfolyam"
    resource: "https://api.example.com/arfolyam"
    method: GET
    headers:
      Authorization: "Bearer !secret api_key"
    value_template: "{{ value_json.rate }}"
    scan_interval: 3600
```

14. Hibakeresés és Validáció

Config check parancsok:

HA OS / Container:

ha core check

HA Core (manuális):

python -m homeassistant --config /path/to/config --check-config

UI-ból:

Developer Tools → YAML → Check Configuration

Log figyelés:

ha logs --follow

vagy: Settings → System → Logs

Gyakori hibák és megoldások:

Hibaüzenet	Ok	Megoldás
found character '\t'	TAB használata	Cseréld 2 space-re
not a valid value for dict value	Boolean probléma	Idézőjelezd az "on"/"off" értékeket
key X is not a valid key	Rossz kulcsnév	Ellenőrizd a dokumentációt
Platform error sensor.X	Integráció hiba	Nézd a logot részletesen
Duplicate key: Y	Dupla kulcs	Töröld vagy merge-eld
Package Y is not valid	Package szintaxis hiba	Ellenőrizd az indentációt

15. YAML Eszközök és Validátorok

Eszköz	Elérhetőség	Leírás
Online YAML Validator	yamllint.com	Gyors szintaxis ellenőrzés online
VS Code YAML extension	Red Hat YAML ext.	IntelliSense + valós idejű validáció
yamllint (CLI)	pip install yamllint	Lokális YAML lint, CI-ba integrálható
CodeBeauty (Android)	Play Store	Szerkesztés mobilon
HA Add-on: Studio Code Server	Beépített HA add-on	Teljes VS Code HA-ban YAML támogatással
HA Add-on: File Editor	Beépített HA add-on	Egyszerű beépített szerkesztő

16. YAML vs UI — Döntési Mátrix

Mit akarsz	UI	YAML
Egyszerű automatizáció	☐	☐
Komplex automatizáció (choose, repeat, parallel)	☒	☐
Egyszerű sensor (MQTT, REST)	☐	☐
Template sensor / binary_sensor	☐	☐
Scene	☐	☐
Script paraméterekkel (fields)	☐	☐
Egyéni entity customize	☐	☐
Packages (logikai csoportosítás)	☐	☐
Verziókezelés (Git)	☐	☐
Secrets kezelés	—	☐
Blueprint import	☐	☐

17. Fontos Linkek

Téma	URL
Configuration.yaml főoldal	home-assistant.io/docs/configuration/
YAML szintaxis (hivatalos)	home-assistant.io/docs/configuration/yaml/
Konfiguráció szétbontása	home-assistant.io/docs/configuration/splitting_configuration/
Packages dokumentáció	home-assistant.io/docs/configuration/packages/
Secrets kezelés	home-assistant.io/docs/configuration/secrets/
Automatizáció YAML-ben	home-assistant.io/docs/automation/yaml/
Script szintaxis	home-assistant.io/docs/scripts/
Conditions (feltételek)	home-assistant.io/docs/scripts/conditions/
Template (Jinja2) referencia	home-assistant.io/docs/configuration/templating/
Blueprints	home-assistant.io/docs/blueprint/
Integrációk listája	home-assistant.io/integrations/
YAML style guide (devs)	developers.home-assistant.io/docs/documenting/yaml-style-guide/
HA Community Forum	community.home-assistant.io/
HA Discord	discord.gg/home-assistant
HA GitHub (példa konfig-ok)	github.com/search?q=topic:home-assistant-config

18. Zárszó — Best Practices

- Legyél konzisztens — 2 space indentáció, logikus mappastruktúra minden fájlban
- Használd packages-t — minden eszközcsoportnak saját fájl a [packages/](#) mappában
- Secrets mindenre — soha nem kerülhet ki GitHub-ra jelszó, API kulcs
- Git repo — a teljes [/config](#) mappát verziókövesd, [.gitignore](#)-ba a secrets.yaml
- Unique ID — minden entity/adatbázisba kerülő dolog kapjon egyedi ID-t
- [alias](#) mező — minden automatizációnak és scriptnek legyen olvasható neve
- Teszteld — [Check Configuration](#) minden változtatás előtt és után
- Fokozatosság — ne egyszerre változtass 100 sort, kis lépésekben haladj
- Backup — mentsd le a konfigurációt mielőtt nagyot módosítasz
- Dokumentáld — kommenteld a nem triviális részeket <#>-tel

Remélem ez az útmutató segít magabiztosan használni a HA YAML konfigurációt. Ha bármi kérdésed van, szólj bátran!

— Hermes Agent, 2026. május